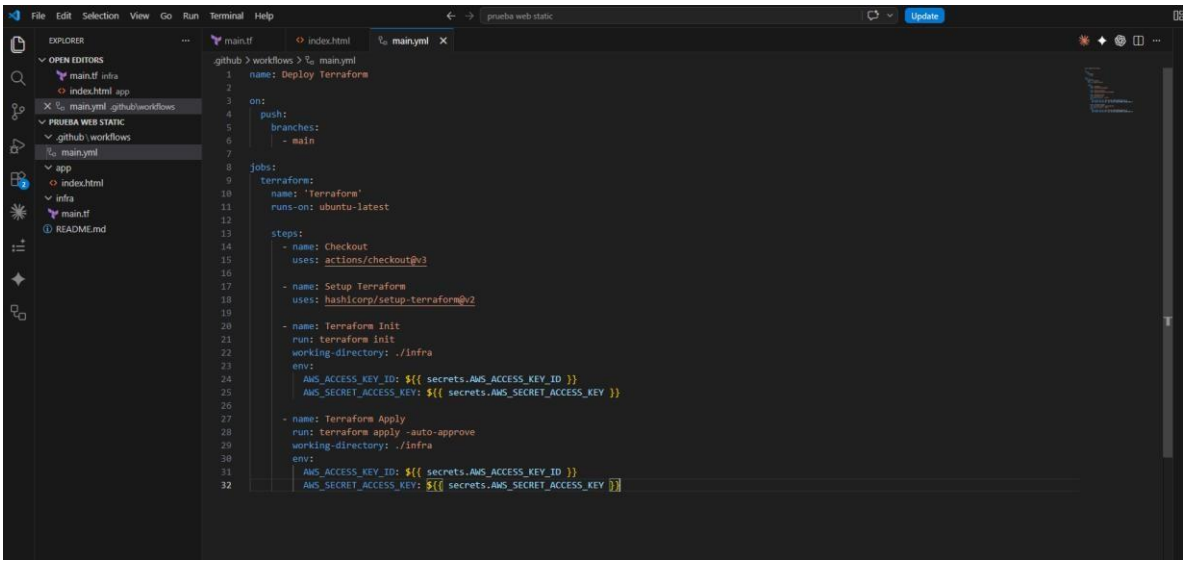
Bitácora de Solución de Errores: Despliegue Automatizado con Terraform y GitHub Actions.

Imagen 1: El primer tropiezo (Error de caracteres)

Aquí se observa el primer error crítico durante la inicialización de Terraform. El sistema detectó un `Invalid character` (carácter inválido) debido a un problema de codificación al copiar el código. Esto nos enseñó que en la Infraestructura como Código (IaC), incluso un espacio invisible puede detener todo el despliegue.



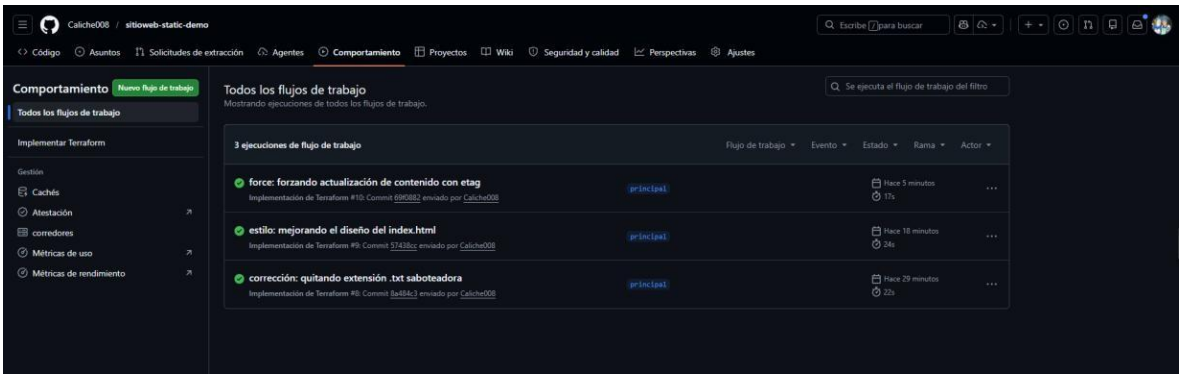
En esta captura vemos el error no such file or directory. Terraform intentaba subir el archivo index.html, pero no lo encontraba en la carpeta app. Esto sucedió porque el archivo tenía una extensión oculta (.txt), lo que impedía que el automatismo lo reconociera correctamente.

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Despliegue Exitoso - Caliche</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #1e3c72 0%, #2a5298 100%);
            height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            margin: 0;
            color: white;
        }
        .card {
            background: rgba(255, 255, 255, 0.1);
            backdrop-filter: blur(10px);
            padding: 3rem;
            border-radius: 20px;
            border: 1px solid rgba(255, 255, 255, 0.2);
            text-align: center;
            box-shadow: 0 8px 32px rgba(0,0,0,0.3);
        }
        h1 { margin: 0; font-size: 2.5rem; color: #00d4ff; }
        p { font-size: 1.2rem; opacity: 0.9; }
        .status {
            display: inline-block;
            margin-top: 20px;
            padding: 10px 20px;
            background: #00ff88;
            color: #1e3c72;
            font-weight: bold;
            border-radius: 50px;
            text-transform: uppercase;
            font-size: 0.8rem;
        }
    </style>
</head>
<body>
    <div class="card">
        <h1>¡Despliegue Exitoso!</h1>
        <p>Hola Profe, este es el proyecto de <strong>Caliche</strong></p>
        <p>Infraestructura como Código (IaC) funcionando en AWS S3.</p>
        <div class="status">● Sistema Online</div>
    </div>
```

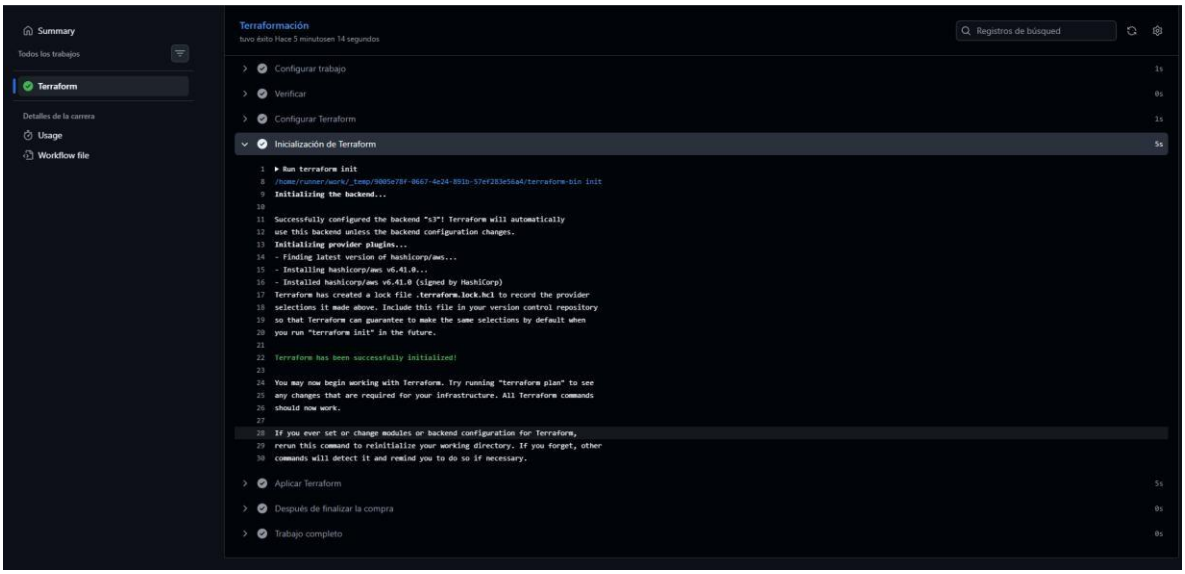
Error de Access Denied al intentar aplicar la política del bucket S3. Este fallo ocurrió porque AWS bloquea por defecto el acceso público. Aprendi que es necesario configurar una dependencia (depends_on) para asegurar que el bloqueo se desactive antes de intentar publicar la política de lectura

```
provider "aws" {
  region = "us-east-1"
}

terraform {
  backend "s3" {
    bucket  = "caliche-remotestate-pro"
    key     = "caliche-proyecto/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
  }
}

resource "aws_s3_bucket" "web" {
  bucket = "caliche-web-static-2026-pro"
}

resource "aws_s3_bucket_website_configuration" "web_config" {
  bucket = aws_s3_bucket.web.id
  index_document {
    suffix = "index.html"
  }
}

# Desbloqueamos el acceso público
resource "aws_s3_bucket_public_access_block" "web_public" {
  bucket = aws_s3_bucket.web.id

  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls      = false
  restrict_public_buckets = false
}

# La política ahora espera a que el bloqueo se quite (Esto quita el error 403)
resource "aws_s3_bucket_policy" "public_read" {
  depends_on = [aws_s3_bucket_public_access_block.web_public]

  bucket = aws_s3_bucket.web.id
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Effect    = "Allow",
      Principal = "*",
      Action    = "s3:GetObject",
      Resource  = "${aws_s3_bucket.web.arn}/*"
    }]
  })
```

Vista del archivo main.tf final. Se aplicaron las correcciones de rutas relativas usando ../app/index.html y se estructuraron los bloques de recursos siguiendo las mejores prácticas de Terraform para asegurar un despliegue fluido y sin errores de acceso.



Confirmación final en GitHub Actions. Tras corregir la extensión del archivo y las dependencias de red, el pipeline se completó con éxito (Check verde). El resultado es una infraestructura automatizada que despliega una página web en Amazon S3 de forma totalmente desatendida



Errores en la implementación

Este fue el error más importante. Windows, por defecto, oculta las extensiones. Tú creaste un archivo de texto y le pusiste "index.html", pero por detrás se llamaba index.html.txt.

- **Lección:** En programación, la extensión es la identidad del archivo. Si el nombre no es exacto, el código no lo encuentra.

2. Caracteres invisibles (Copy-Paste)

Al principio, Terraform te daba errores de Invalid character en líneas que parecían estar bien. Esto pasa porque al copiar código de la web o de chats, a veces se vienen espacios especiales (UTF-8) que el ojo humano no ve, pero que para Terraform son basura.

- **Lección:** Si un código se ve bien pero falla, mejor borrar todo y volver a escribirlo o pegarlo en "texto plano".

3. El orden de los factores sí altera el producto (S3 Public Access)

Intentaste poner una política pública en S3 antes de decirle a AWS que "abriera el candado". AWS bloquea por seguridad cualquier política pública si el Public Access Block está activo.

- **Lección:** En la nube, primero desbloqueas la seguridad (Public Access Block) y luego aplicas la regla (Policy). Por eso usamos el depends_on.

4. Nombres de Bucket repetidos

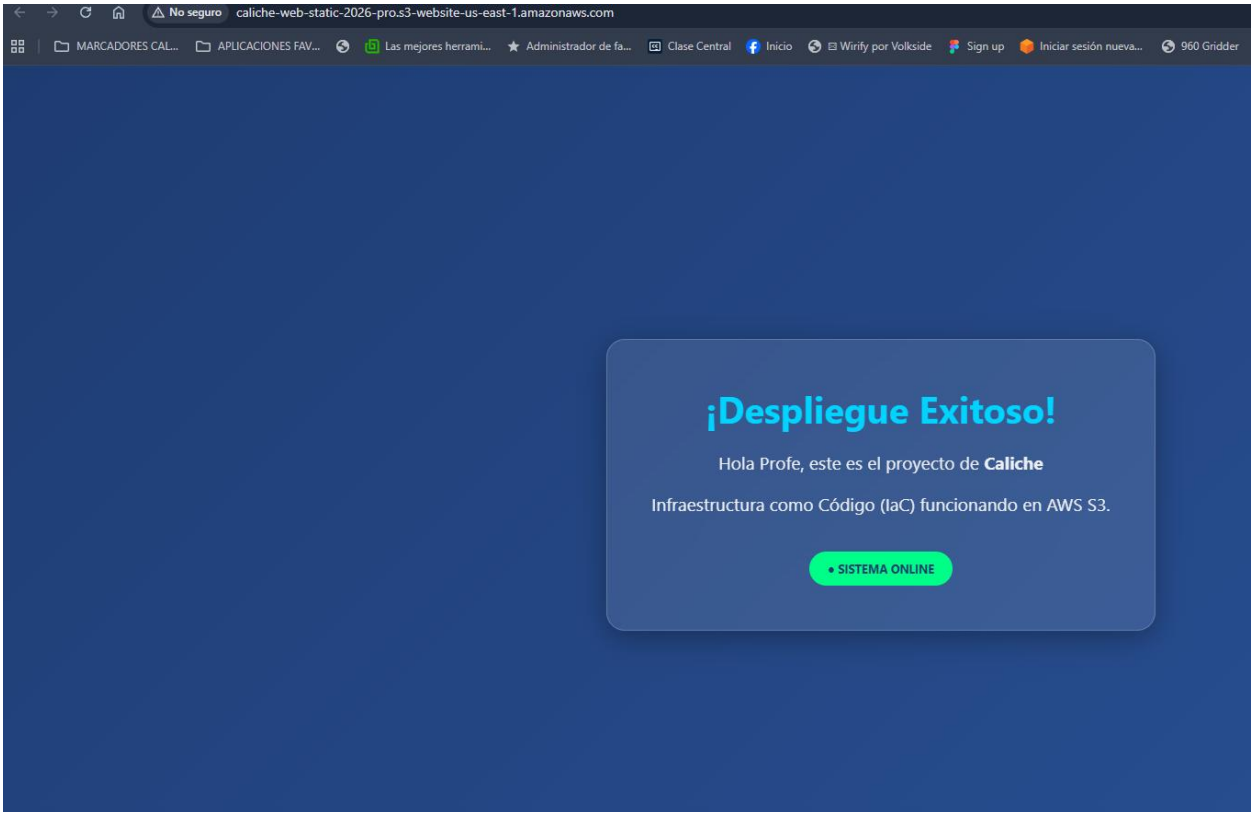
Al principio usamos nombres comunes. En AWS S3, el nombre del bucket tiene que ser **único en todo el mundo**. Si alguien en China ya usó caliche-web-static, tú ya no puedes usarlo.

- **Lección:** Siempre añade prefijos, sufijos o números únicos a tus recursos de nube.

5. Rutas relativas (../)

Hubo un momento de confusión sobre dónde estaba parado el "robot" de GitHub. Al estar el código dentro de infra/, para llegar a app/ había que decirle: *"Sube un nivel y luego entra al otro"*.

- **Lección:** La estructura de carpetas es el mapa del tesoro para tus herramientas de automatización.


No seguro
caliche-web-static-2026-pro.s3-website-us-east-1.amazonaws.com
MARCADORES CAL...
APLICACIONES FAV...
Las mejores herrami...
Administrador de fa...
Clase Central
Inicio
Wirify por Volkside
Sign up
Iniciar sesión nueva...
960 Gridder
¡Despliegue Exitoso!
Hola Profe, este es el proyecto de Caliche
Infraestructura como Código (IaC) funcionando en AWS S3.
• SISTEMA ONLINE